SECURITY



Secure Key Distribution Protocol for SoC/SiP Hardware blocks

Craig Barner *Senior Distinguished Engineer, Marvell*

Darren Lasko *Principal Engineer, NVIDIA*

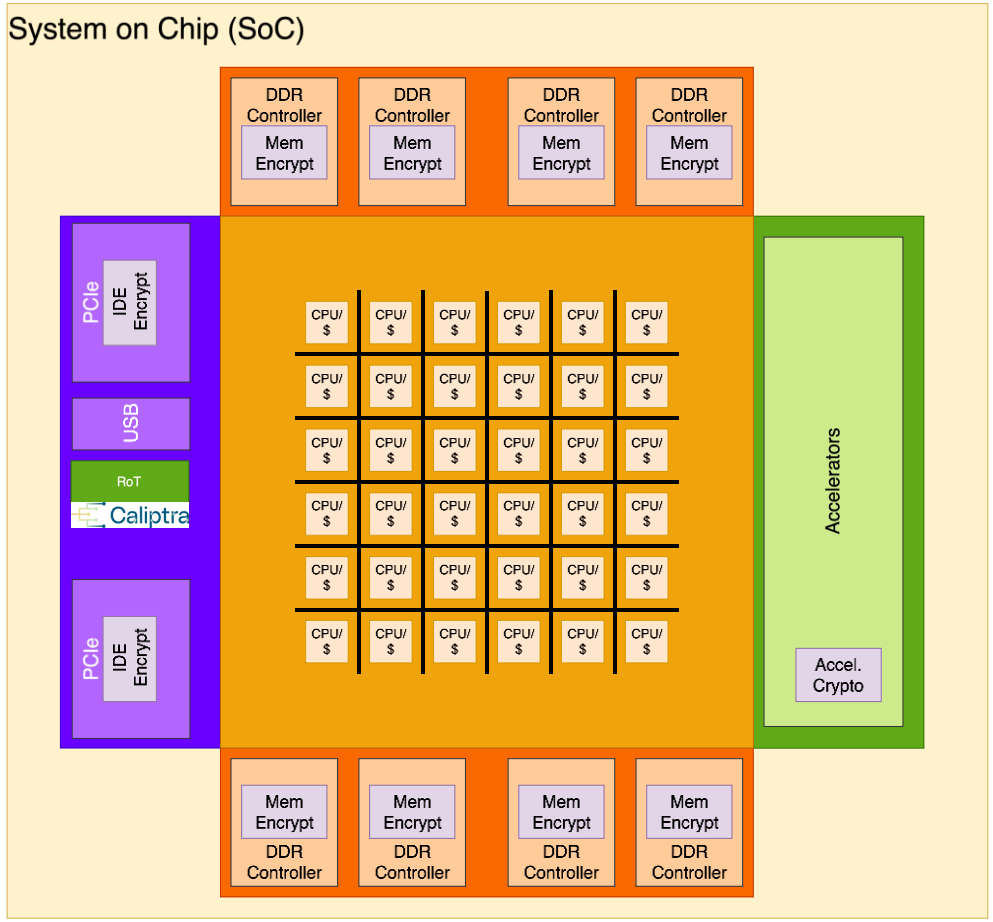
Forward-looking statements

Except for statements of historical fact, this presentation contains forward-looking statements (within the meaning of the federal securities laws) including statements related to future revenue, future earnings, and the success of our product releases that involve risks and uncertainties. Words such as “anticipates,” “expects,” “intends,” “plans,” “projects,” “believes,” “seeks,” “estimates,” “can,” “may,” “will,” “would” and similar expressions identify such forward-looking statements. These statements are not guarantees of results and should not be considered as an indication of future activity or future performance. Actual events or results may differ materially from those described in this presentation due to a number of risks and uncertainties.

Forward-looking statements are only predictions and are subject to risks, uncertainties and assumptions that are difficult to predict, including those described in the “Risk Factors” section of our Annual Reports on Form 10-K, Quarterly Reports on Form 10-Q and other documents filed by us from time to time with the SEC. Forward-looking statements speak only as of the date they are made. You are cautioned not to put undue reliance on forward-looking statements, and no person assumes any obligation to update or revise any such forward-looking statements, whether as a result of new information, future events or otherwise.

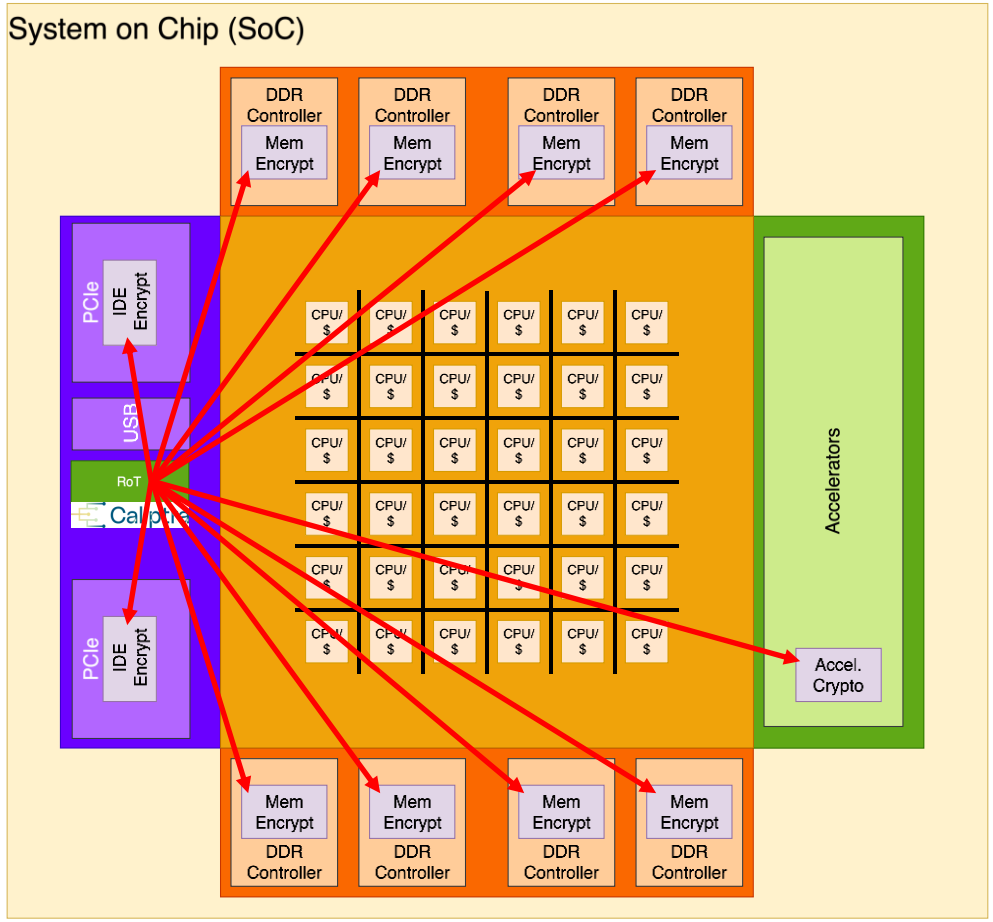
© 2026 Marvell. All rights reserved.

Recap: The Key Management Problem



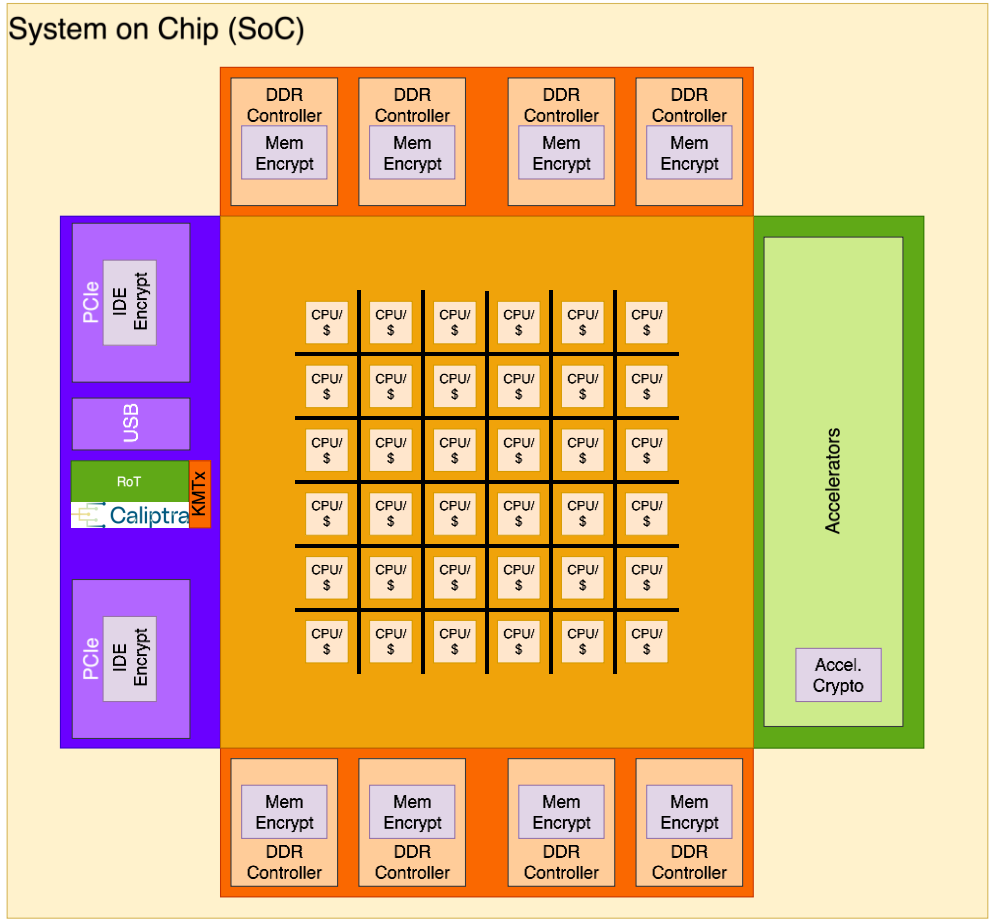
- Every HW block that encrypts data needs key management
  - Creation, verification, zeroization, rotation
- Keys may be managed ad-hoc today
  - By the HW block itself
  - Programmed by system software
  - By a dedicated Security Processor
  - ➢ *Current solutions are vendor-unique*
- Must stay correct across space (components) and time (lifecycle)
- At SoC scale, this is now a system-wide problem
  - A System-in-Package (SiP) containing chiplets from multiple vendors adds another layer of challenge!

Recap: Impact of point-to-point buses



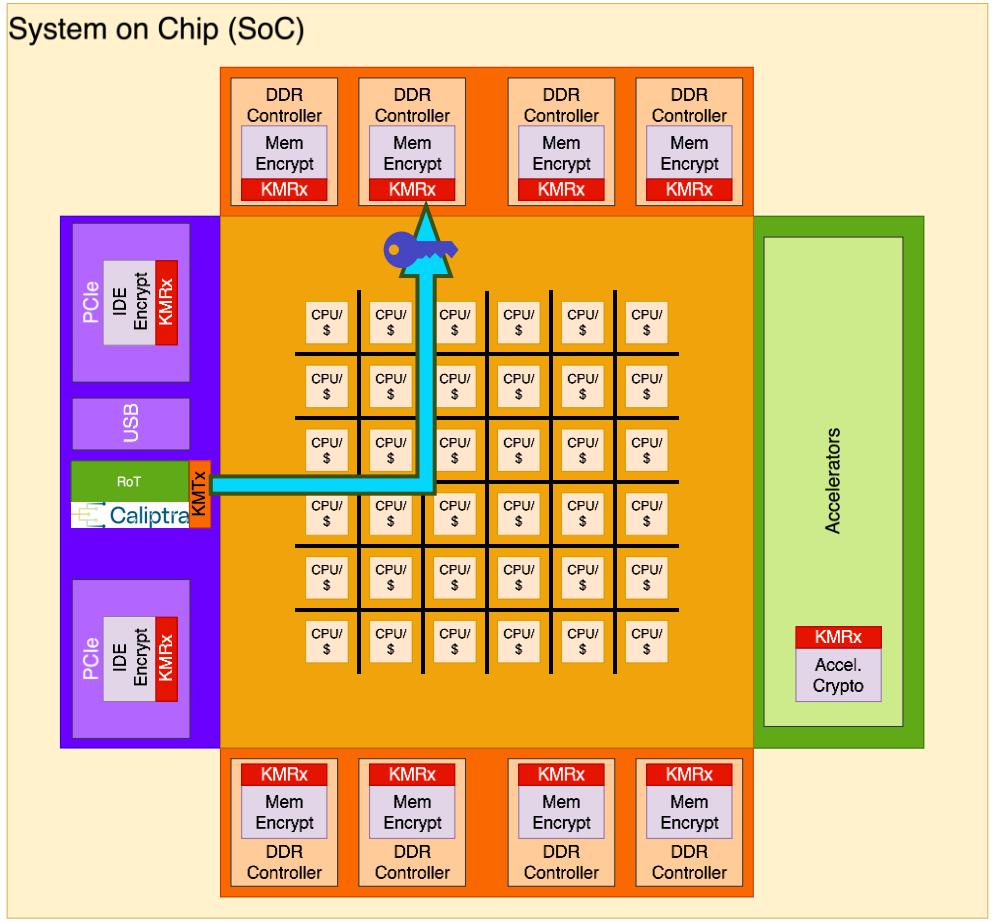
- Shared buses pose a security risk
- Using point-to-point “private” buses for keys is not scalable in large designs
- *Can we create a virtual private bus?*

Recap: Proposed solution



- KMTx: Key Management Transmitter
  - A feature of the RoT (e.g. Caliptra)
  - Can be implemented in firmware

Recap: Proposed solution



- KMTx: Key Management Transmitter
  - A feature of the RoT (e.g. Caliptra)
  - Can be implemented in firmware
- KMRx: Key Management Receiver
  - A small, memory-mapped IP block that is paired with one or more “Downstream Crypto Engines” (DCEs)
- KMTx and KMRx share a Session Key
- KMTx securely transmits Data Keys to KMRx using Session Key over *shared bus/fabric*

Progress since last year

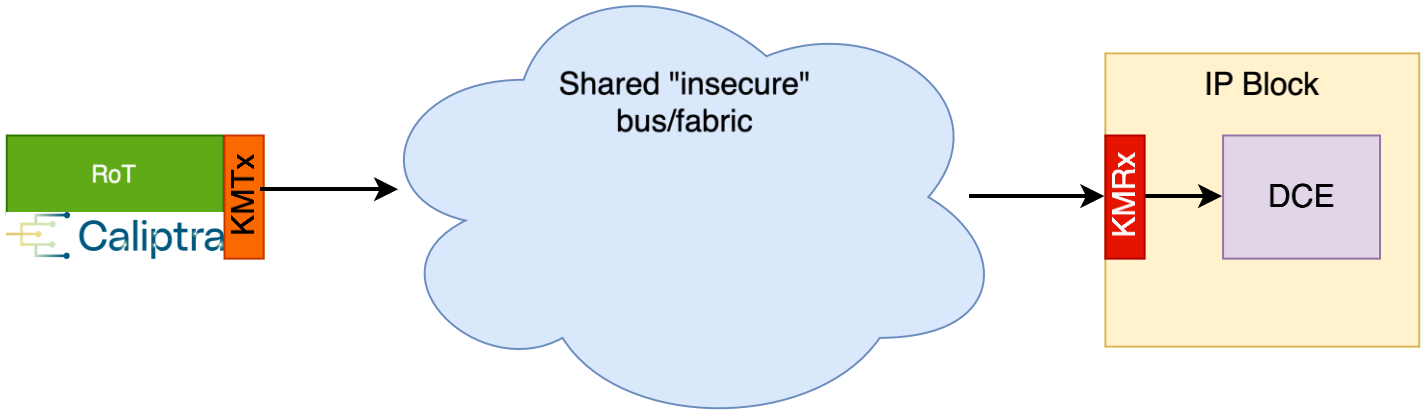
BREAKING | **Spec v0.7 is out**

- We now have a 0.7 spec!
- The “**KMRx MMIO and Protocol Architecture Specification**” is now available in the OCP Security Hub Google Drive folder
- The specification defines:
  - The protocol for KMTx-KMRx communication
  - The format of the encrypted and integrity-protected payload
  - The MMIO address space of the KMRx
  - The actions that KMRx takes in response to payloads it receives

0.7

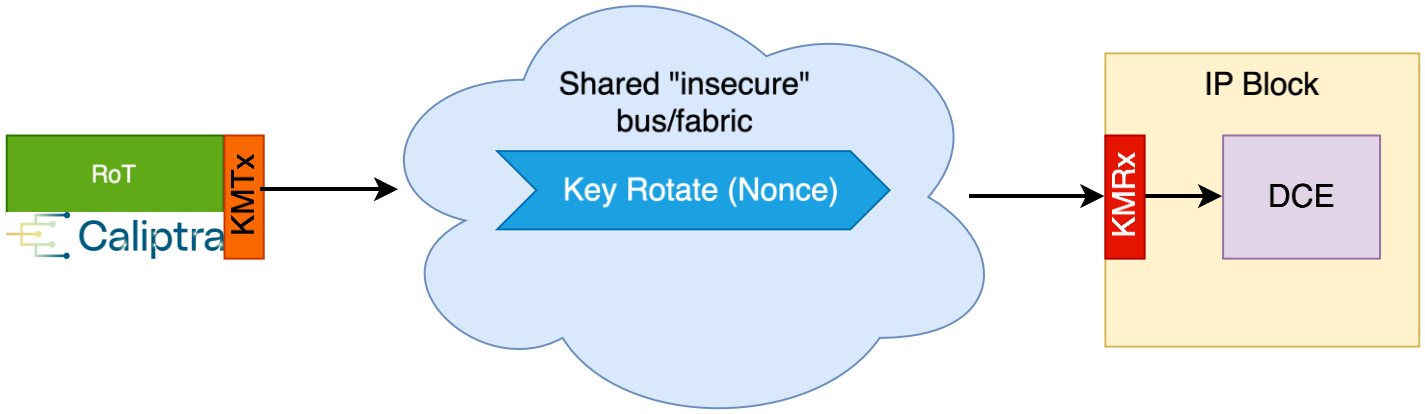
KMRx MMIO and Protocol Architecture Specification

High-Level Overview of Architecture



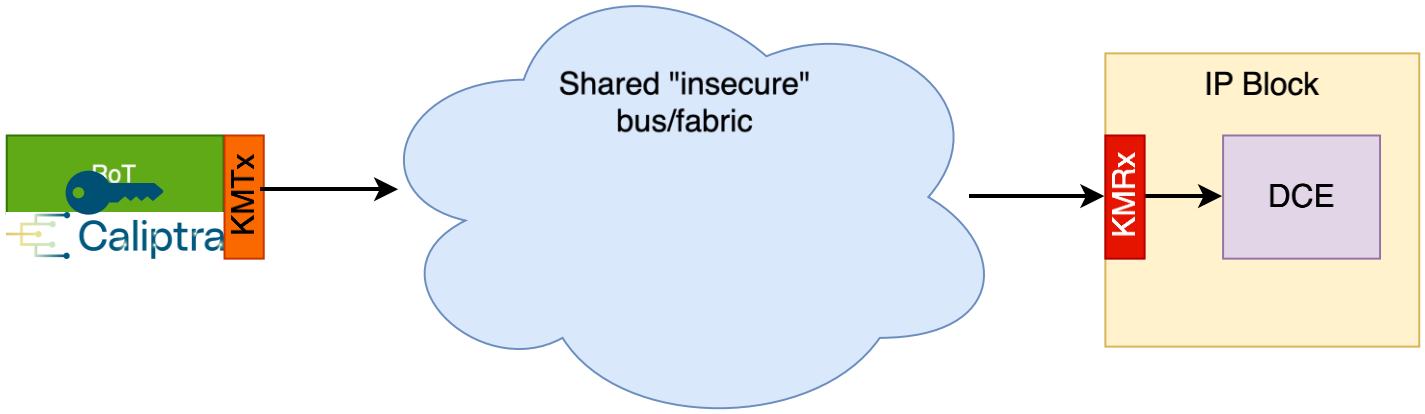
1. KMTx and KMRx share Initial Pairing Key at reset
   - Method is currently implementation-defined
   - WG may define some methods, such as private secure serial bus or by using RTL keys

High-Level Overview of Architecture



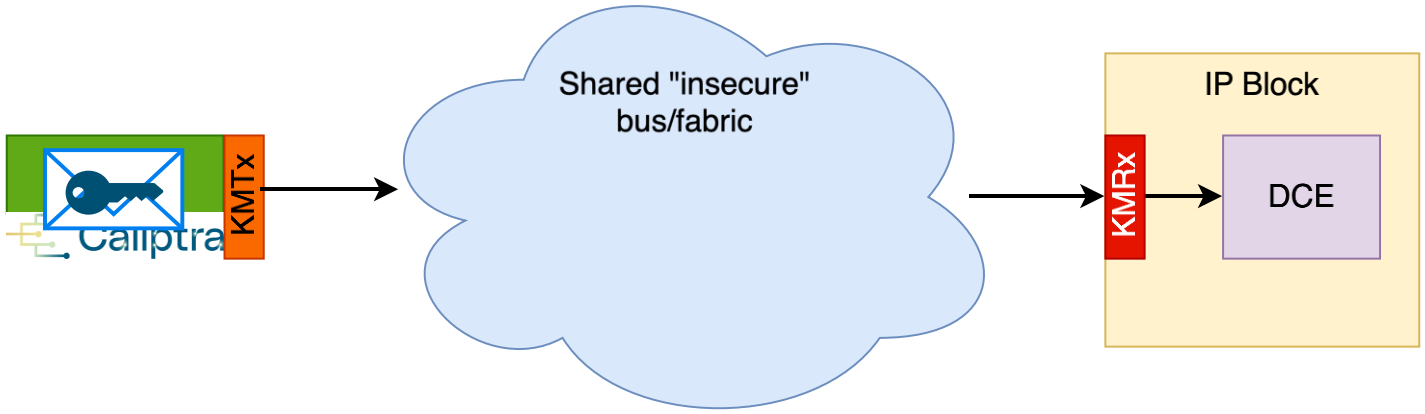
1. KMTx and KMRx share Initial Pairing Key at reset
   - Method is currently implementation-defined
   - WG may define some methods, such as private secure serial bus or by using RTL keys
2. RoT issues a **Key Rotation** command (with random nonce) to KMRx to establish the shared Session Key (derived from Initial Pairing Key + nonce)

High-Level Overview of Architecture



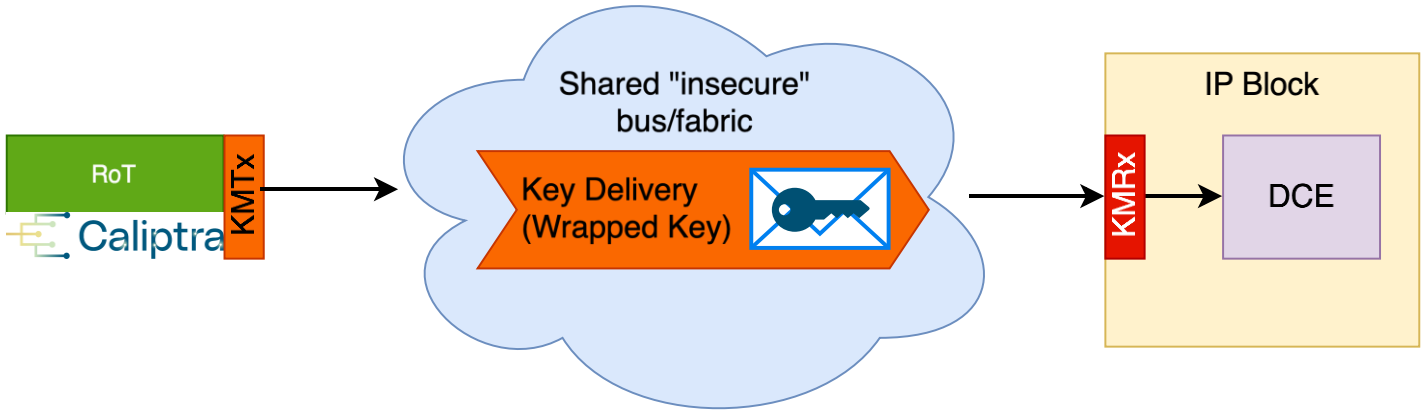
1. KMTx and KMRx share Initial Pairing Key at reset
   - Method is currently implementation-defined
   - WG may define some methods, such as private secure serial bus or by using RTL keys
2. RoT issues a **Key Rotation** command (with random nonce) to KMRx to establish the shared Session Key (derived from Initial Pairing Key + nonce)
3. RoT generates a Data Key for sending to the DCE

High-Level Overview of Architecture



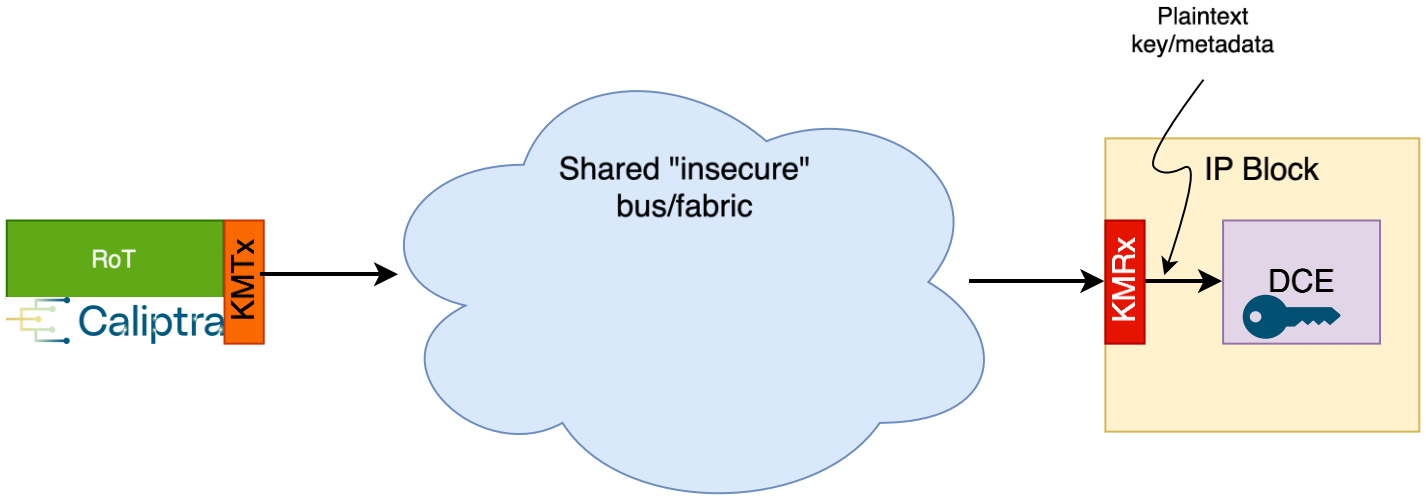
1. KMTx and KMRx share Initial Pairing Key at reset
   - Method is currently implementation-defined
   - WG may define some methods, such as private secure serial bus or by using RTL keys
2. RoT issues a **Key Rotation** command (with random nonce) to KMRx to establish the shared Session Key (derived from Initial Pairing Key + nonce)
3. RoT generates a Data Key for sending to the DCE
4. RoT "wraps" (AES-GCM encrypts) the Data Key and its metadata using the Session Key for the DCE's KMRx (payload contains the **Key Delivery** command)

High-Level Overview of Architecture



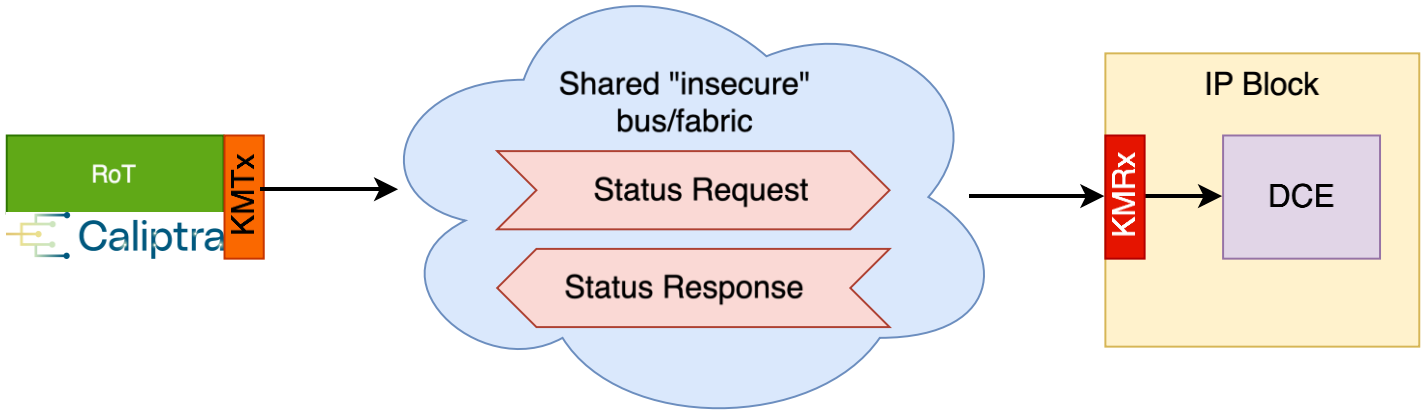
1. KMTx and KMRx share Initial Pairing Key at reset
   - Method is currently implementation-defined
   - WG may define some methods, such as private secure serial bus or by using RTL keys
2. RoT issues a **Key Rotation** command (with random nonce) to KMRx to establish the shared Session Key (derived from Initial Pairing Key + nonce)
3. RoT generates a Data Key for sending to the DCE
4. RoT "wraps" (AES-GCM encrypts) the Data Key and its metadata using the Session Key for the DCE's KMRx (payload contains the **Key Delivery** command)
5. RoT writes the encrypted payload to the MMIO space of the KMRx

High-Level Overview of Architecture



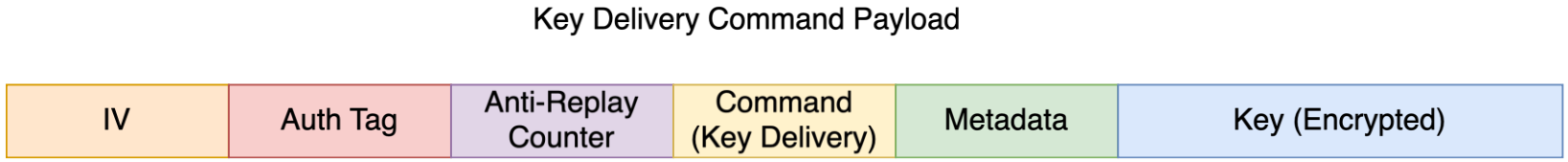
1. KMTx and KMRx share Initial Pairing Key at reset
   - Method is currently implementation-defined
   - WG may define some methods, such as private secure serial bus or by using RTL keys
2. RoT issues a **Key Rotation** command (with random nonce) to KMRx to establish the shared Session Key (derived from Initial Pairing Key + nonce)
3. RoT generates a Data Key for sending to the DCE
4. RoT "wraps" (AES-GCM encrypts) the Data Key and its metadata using the Session Key for the DCE's KMRx (payload contains the **Key Delivery** command)
5. RoT writes the encrypted payload to the MMIO space of the KMRx
6. KMRx "unwraps" (AES-GCM decrypts) the payload, confirms its authenticity, and sends the plaintext key/metadata to the DCE

High-Level Overview of Architecture



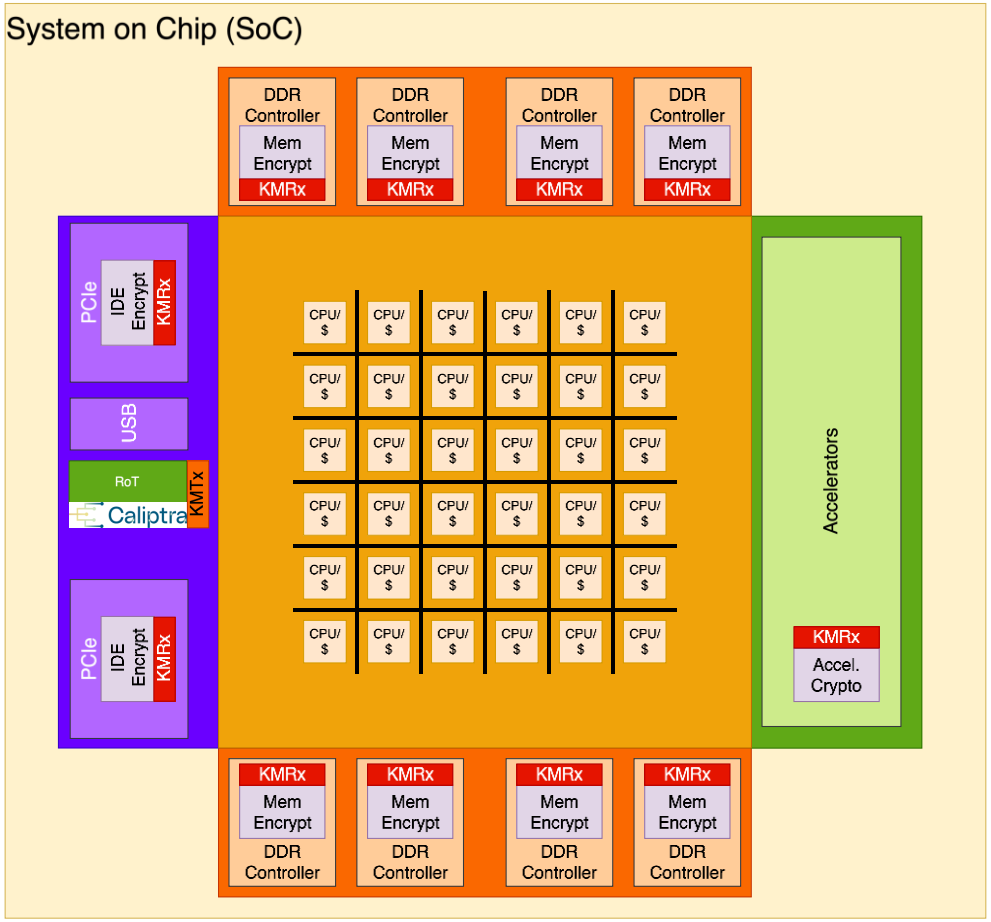
1. KMTx and KMRx share Initial Pairing Key at reset
   - Method is currently implementation-defined
   - WG may define some methods, such as private secure serial bus or by using RTL keys
2. RoT issues a **Key Rotation** command (with random nonce) to KMRx to establish the shared Session Key (derived from Initial Pairing Key + nonce)
3. RoT generates a Data Key for sending to the DCE
4. RoT "wraps" (AES-GCM encrypts) the Data Key and its metadata using the Session Key for the DCE's KMRx (payload contains the **Key Delivery** command)
5. RoT writes the encrypted payload to the MMIO space of the KMRx
6. KMRx "unwraps" (AES-GCM decrypts) the payload, confirms its authenticity, and sends the plaintext key/metadata to the DCE
7. RoT optionally confirms successful key delivery using the **Status Request** command (response is cryptographically-protected to prevent spoofing of success)

KMTx/KMRx Protocol Security Guarantees



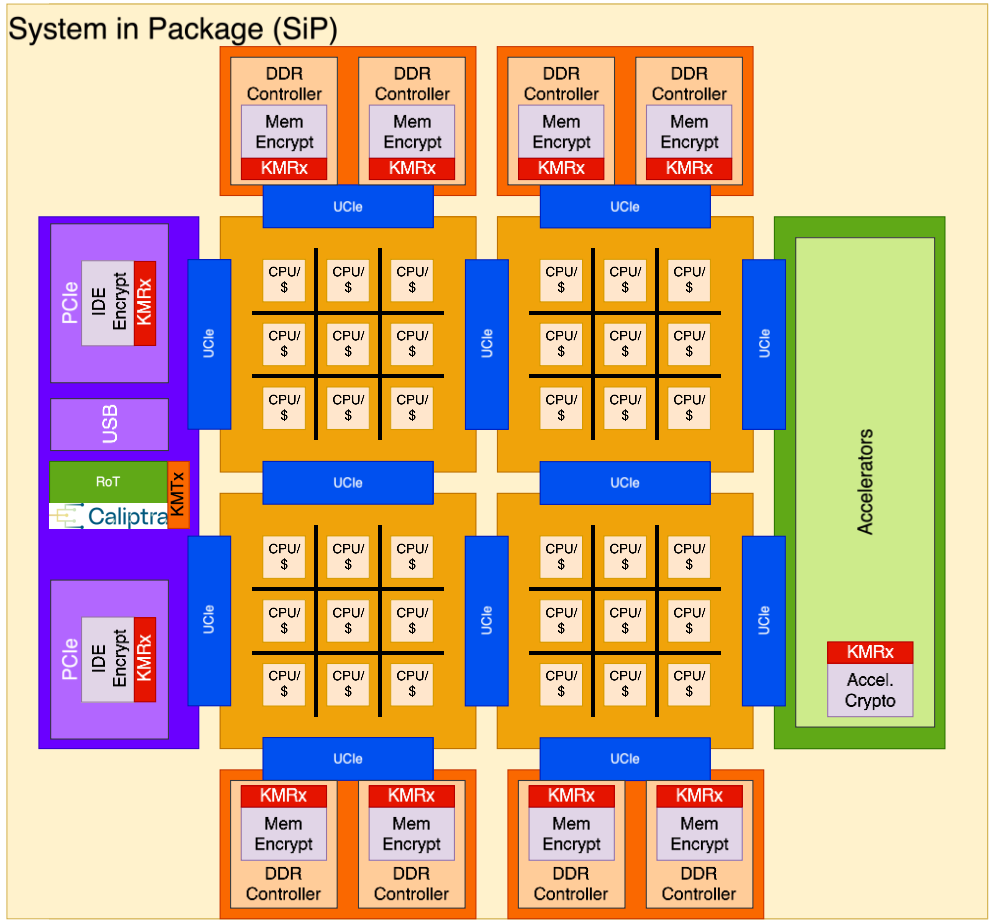
- *Confidentiality* and *integrity* of keys delivered over shared buses/fabrics
  - AES-GCM encrypted/authenticated payloads
- *Anti-replay* and *anti-drop* protection
  - Anti-replay counter in payload ensures that keys cannot be replayed/re-ordered/dropped
  - **Status Request** command can be used for cryptographic proof of delivery
- *Anti-rerouting* protection
  - Different KMRx instances have different session keys, so authentication tag check will fail if a payload is maliciously re-routed to the wrong KMRx

SoC to SiP



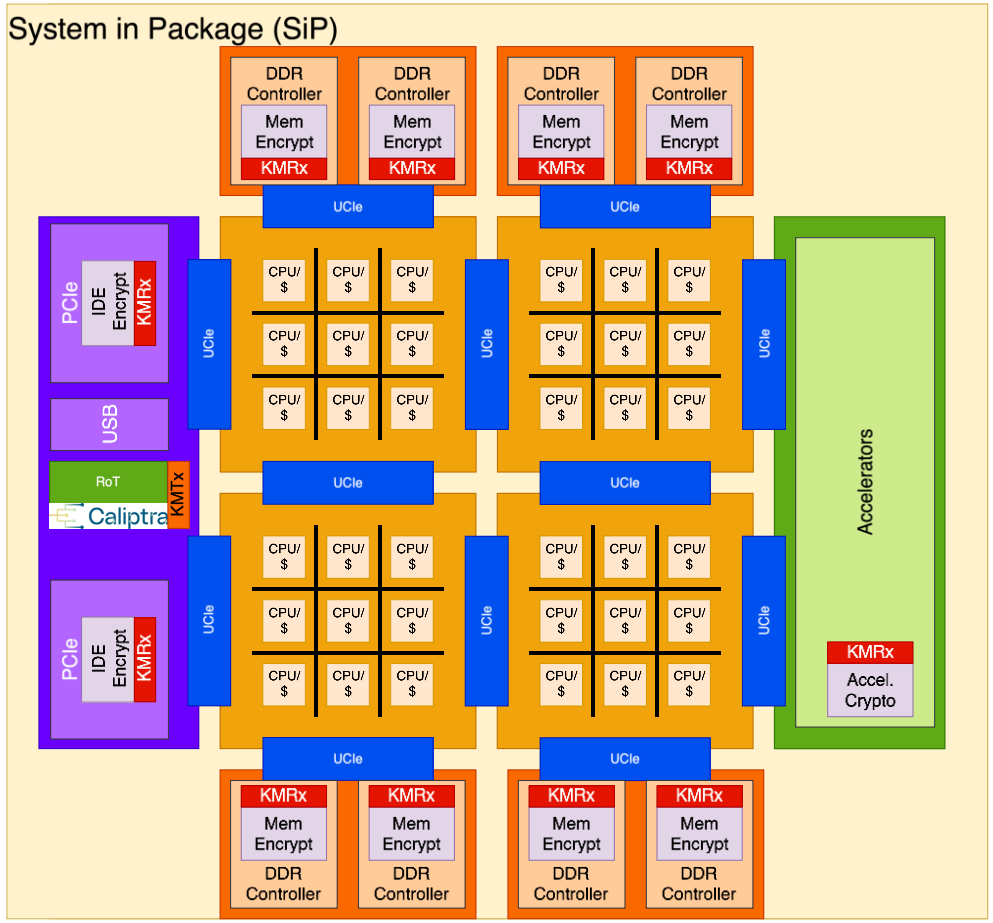
- What happens when your System-on-Chip (SoC, monolithic silicon die)…

SoC to SiP



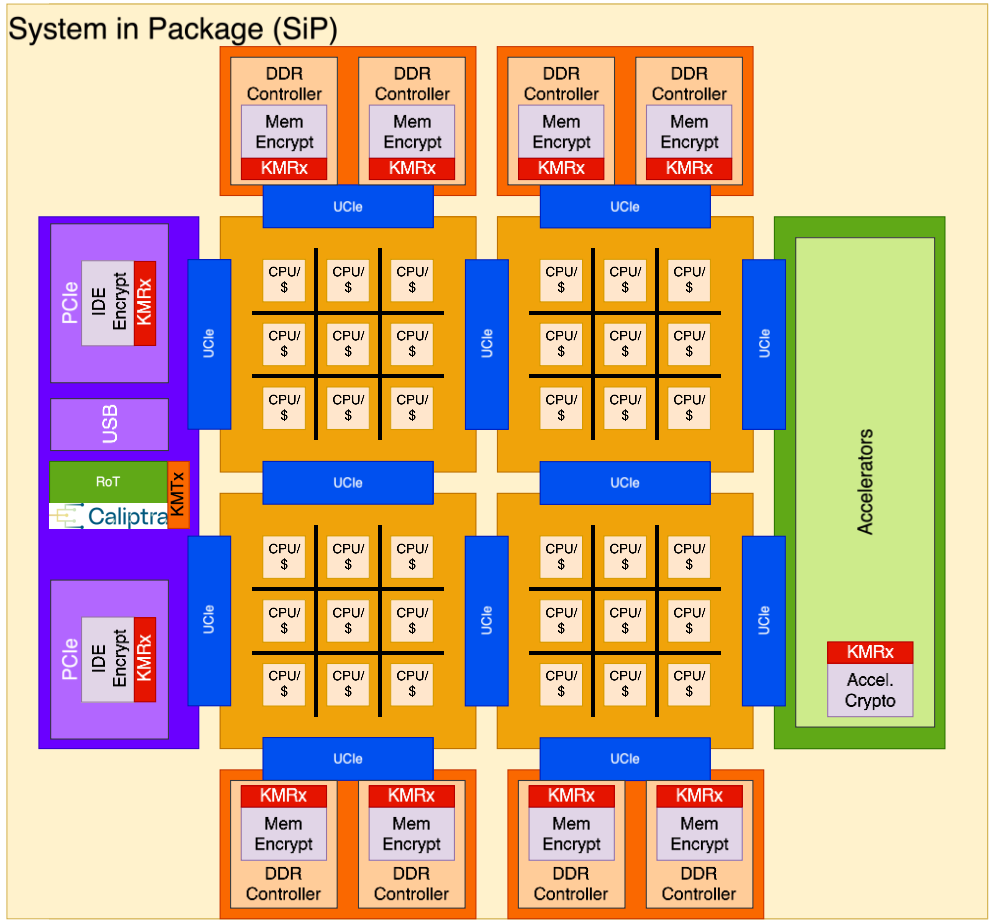
- What happens when your System-on-Chip (SoC, monolithic silicon die)…
- …becomes a System-in-Package (SiP, multiple silicon “chiplets”)?

SoC to SiP



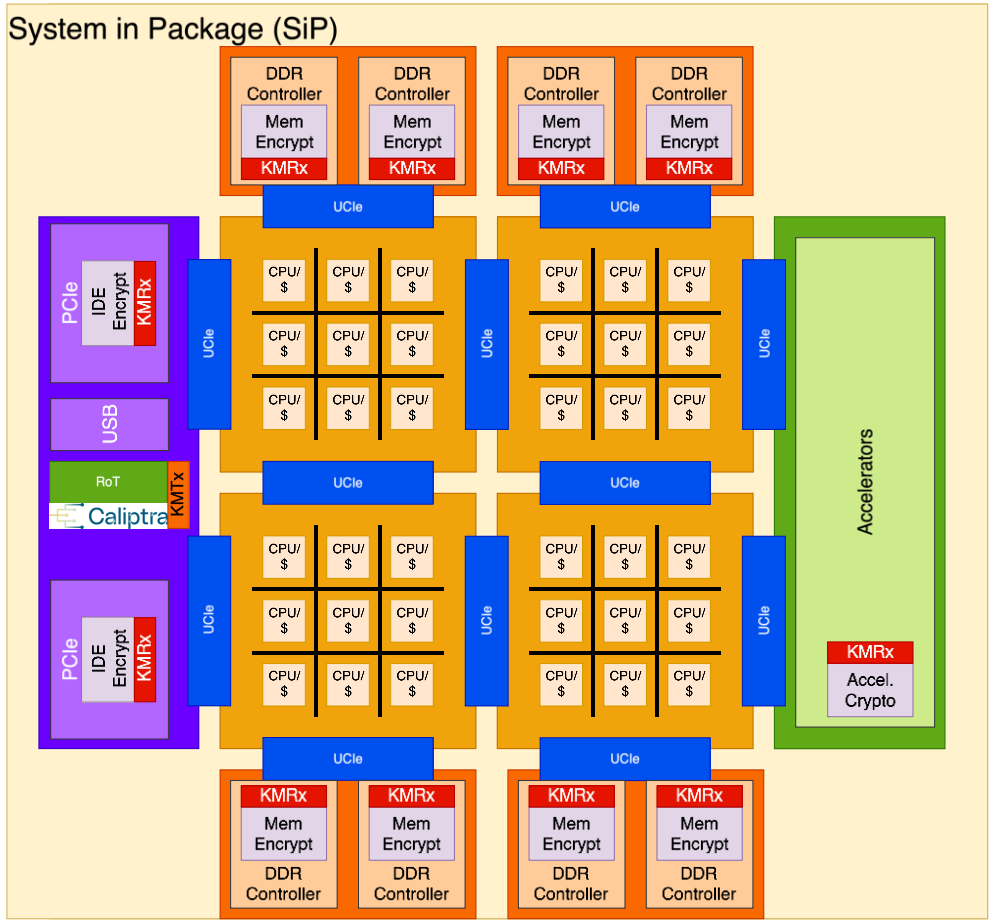
- What happens when your System-on-Chip (SoC, monolithic silicon die)…
- …becomes a System-in-Package (SiP, multiple silicon “chiplets”)?
  - How can you *securely* establish an Initial Pairing Key between the RoT and a KMRx on a different chiplet? (No “secure bus” across chiplet-to-chiplet links)

SoC to SiP



- What happens when your System-on-Chip (SoC, monolithic silicon die)…
- …becomes a System-in-Package (SiP, multiple silicon “chiplets”)?
  - How can you *securely* establish an Initial Pairing Key between the RoT and a KMRx on a different chiplet? (No “secure bus” across chiplet-to-chiplet links)
    - *Especially* when the chiplets are from different silicon vendors

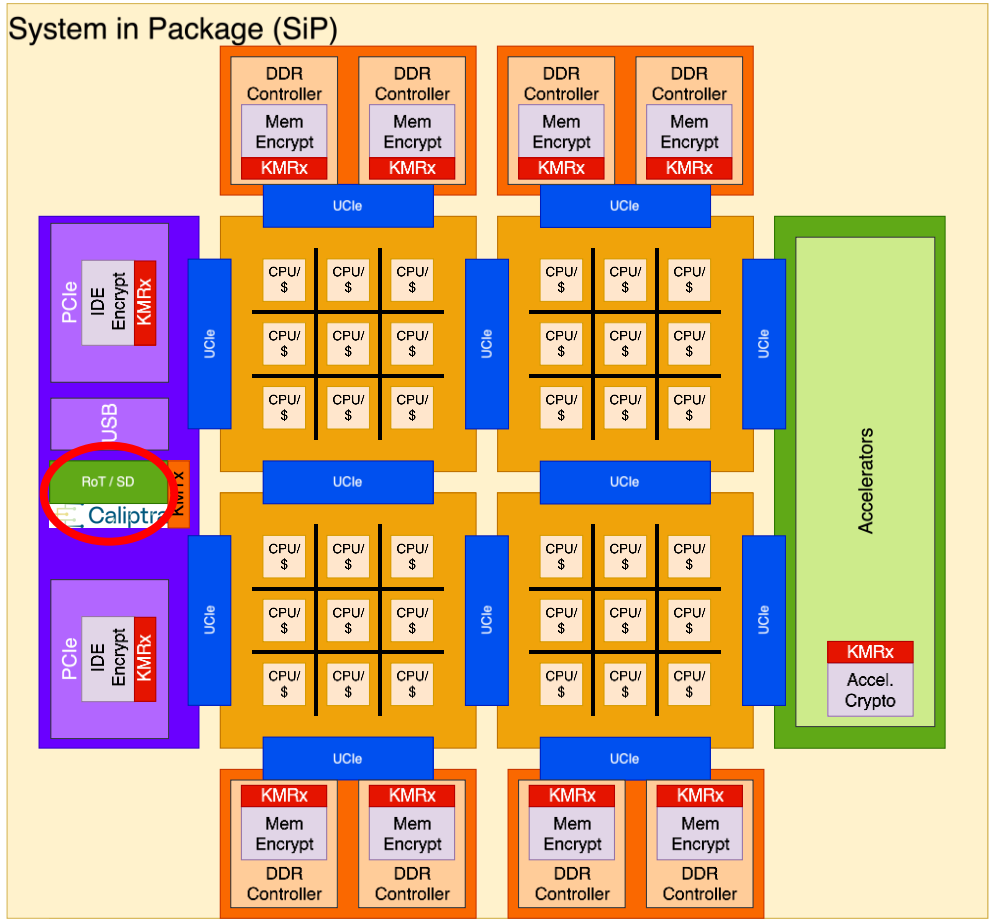
Proposed Solution: Engage with, and leverage the work done in, UCIe



Universal Chiplet Interconnect Express

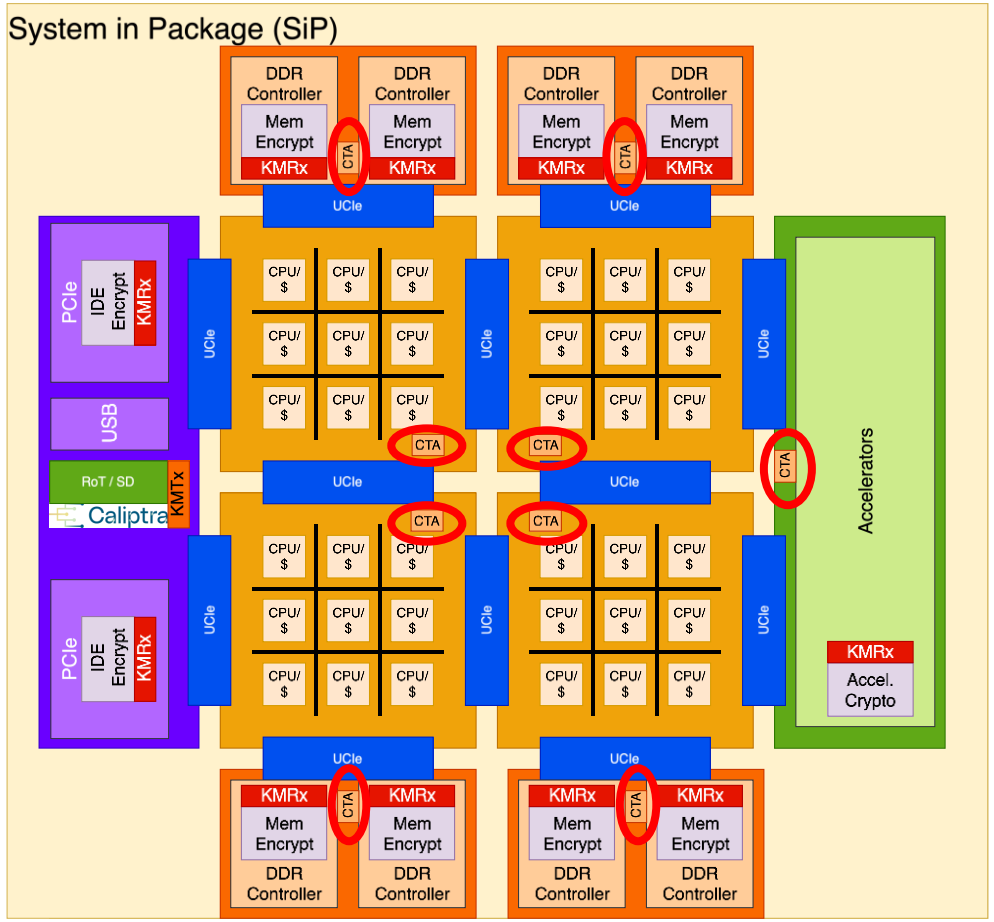
(UCIe) is an open, multi-protocol capable, on-package interconnect standard for connecting multiple dies ("chiplets") on the same package. The primary motivation is to enable a vibrant ecosystem supporting disaggregated die architectures which can be interconnected using UCIe.

Background: UCIe “Security Director” and “Chiplet Trust Agent”



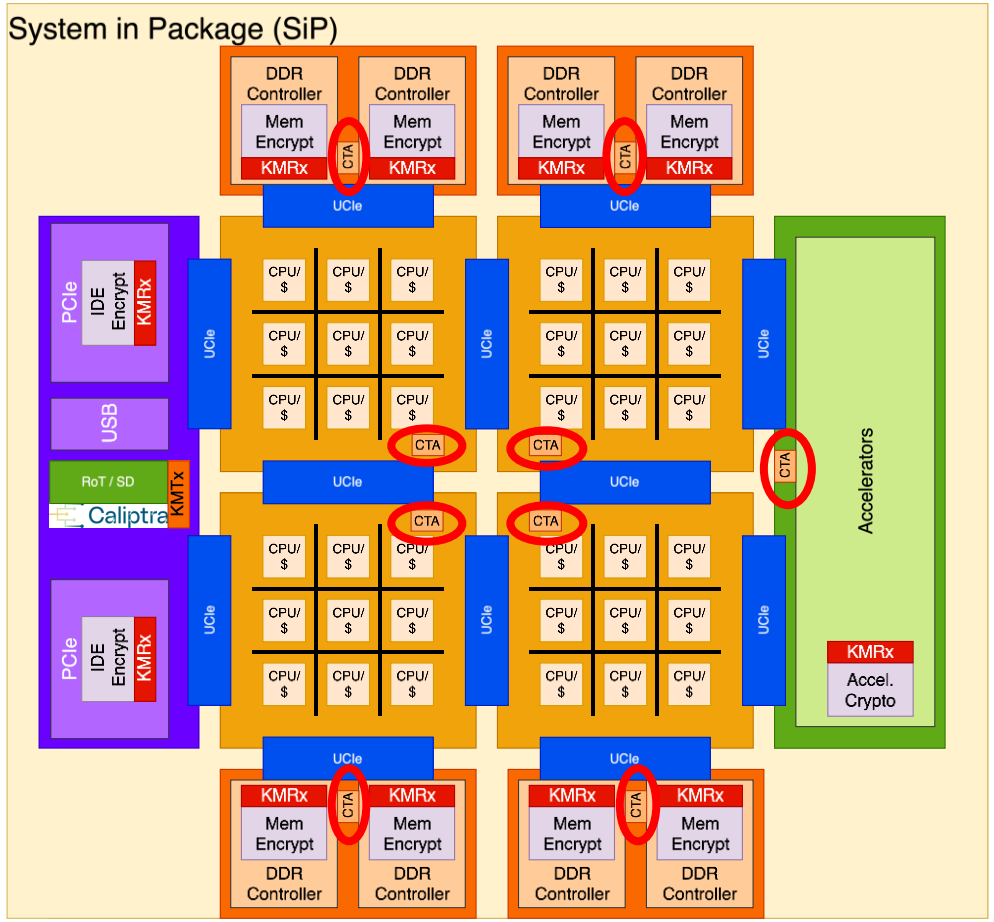
- UCIe defines the concepts of:
  - “Security Director” (SD), which is the main RoT in the SiP

Background: UCIe “Security Director” and “Chiplet Trust Agent”



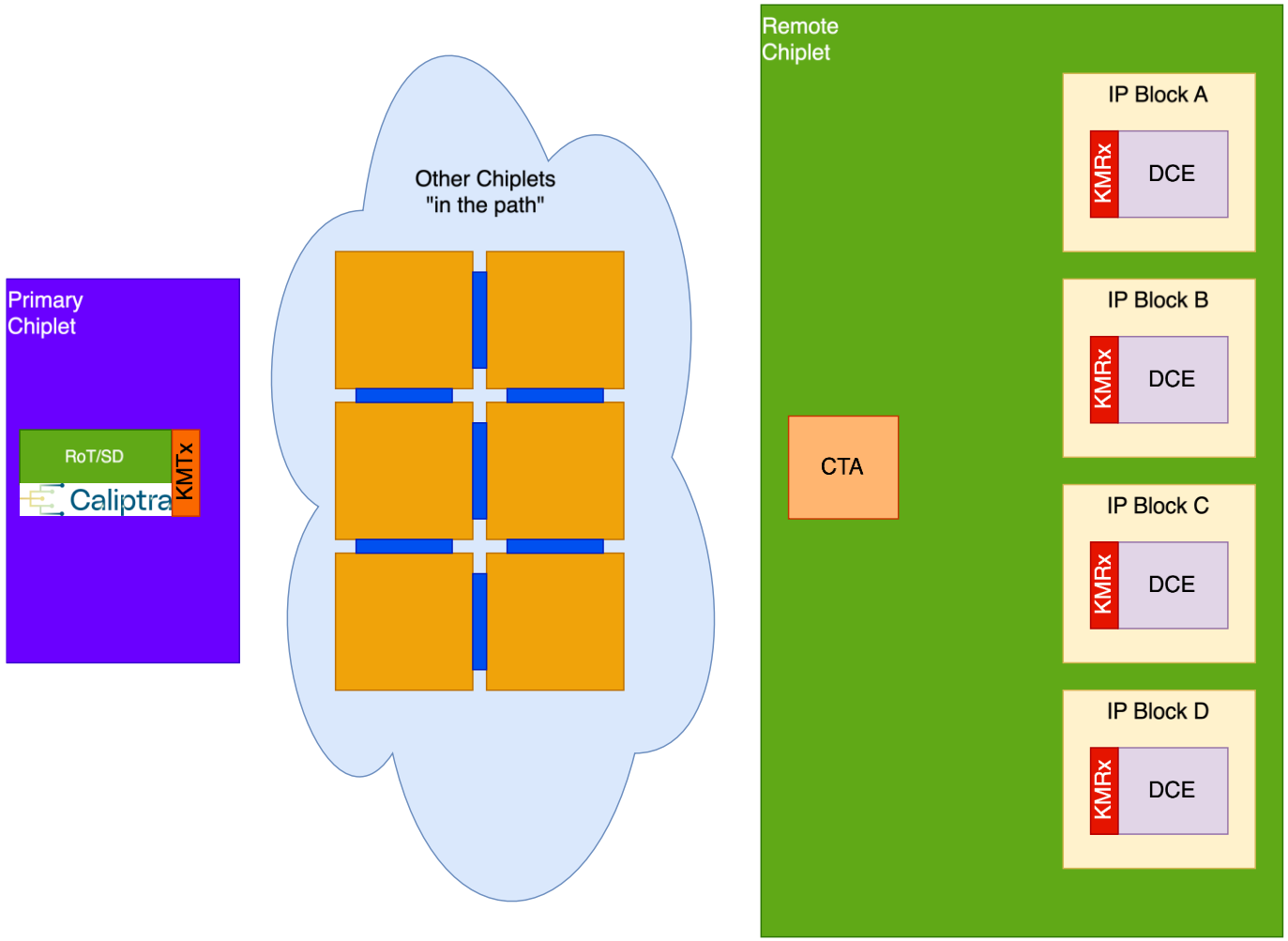
- UCIe defines the concepts of:
  - “Security Director” (SD), which is the main RoT in the SiP
  - “Chiplet Trust Agent” (CTA), which is “*a security-hardened module within a chiplet that provides various security-related capabilities and services*”

Background: UCIe “Security Director” and “Chiplet Trust Agent”



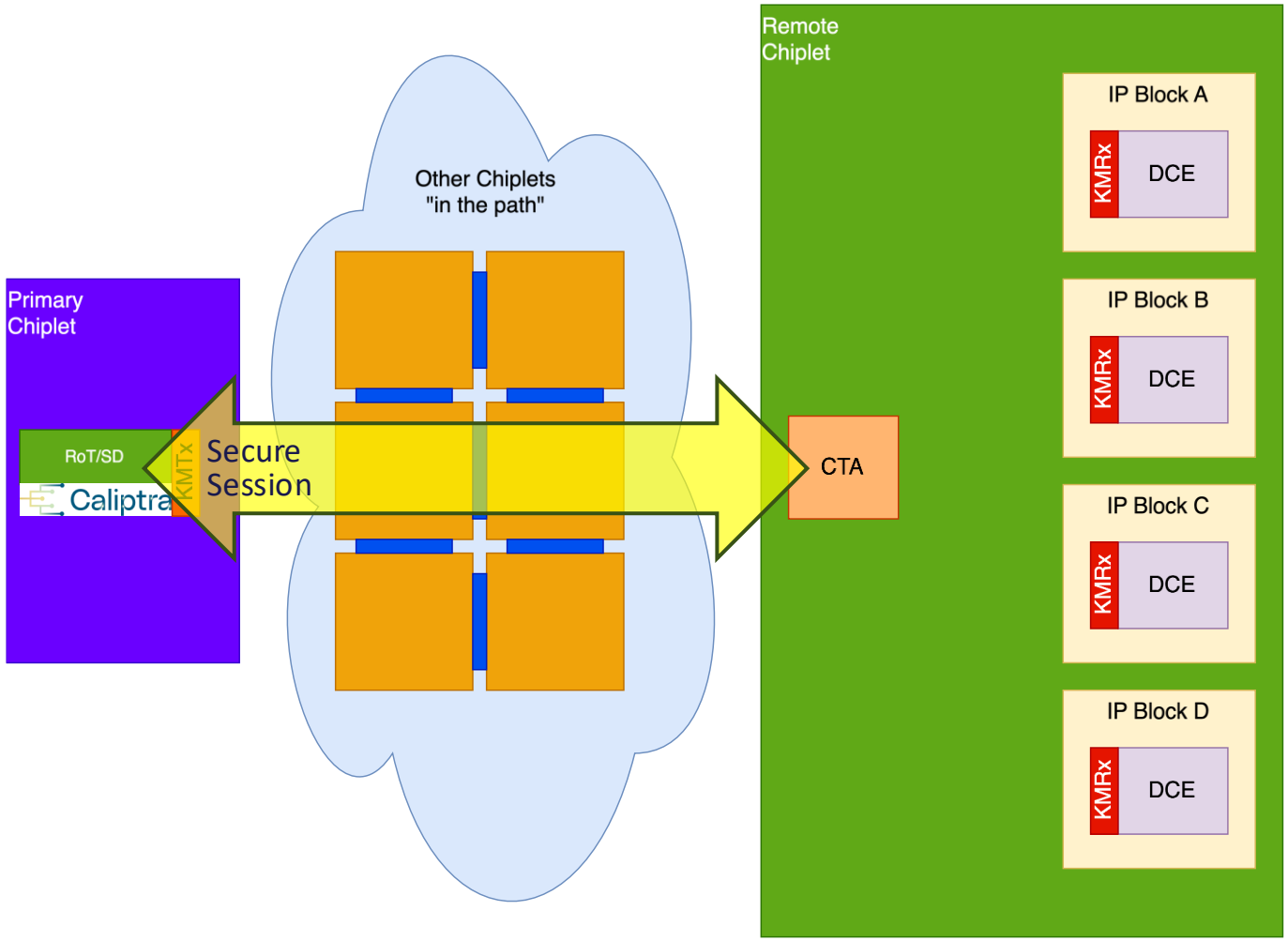
- UCIe defines the concepts of
  - “Security Director” (SD), which is the main RoT in the SiP
  - “Chiplet Trust Agent” (CTA), which is “*a security-hardened module within a chiplet that provides various security-related capabilities and services*”
- During SiP integration, the SD and the CTAs undergo a secure provisioning process to establish cryptographic shared secrets between the SD and each CTA
  - Each CTA gets a unique **CTA_SEED**, used for deriving keys for various CTA features

Proposed CTA feature: KMRx Initial Pairing Key (IPK) Establishment



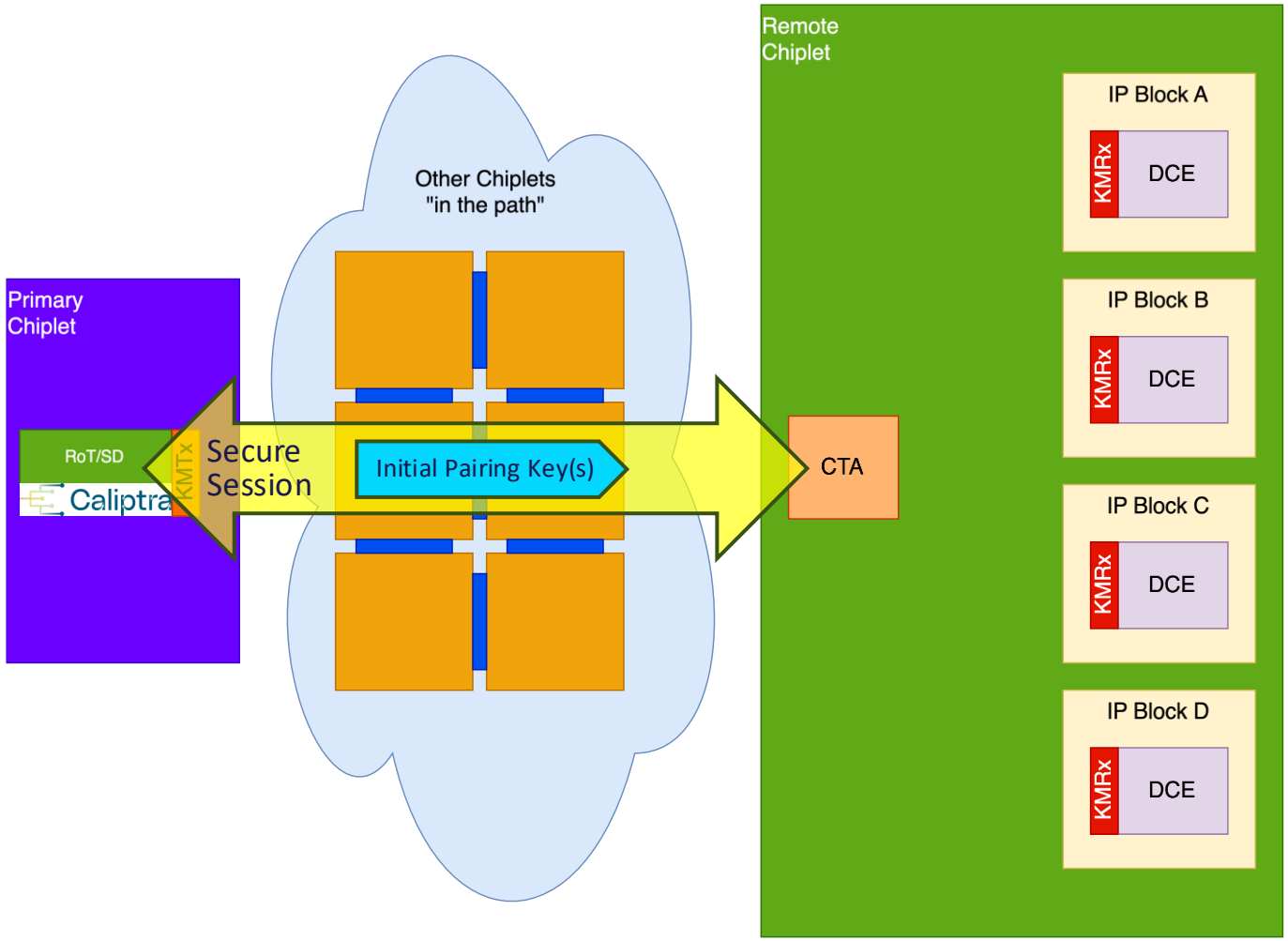
- RoT establishes Initial Pairing Keys to KMRx instances on “remote” chiplets as follows:

Proposed CTA feature: KMRx Initial Pairing Key (IPK) Establishment



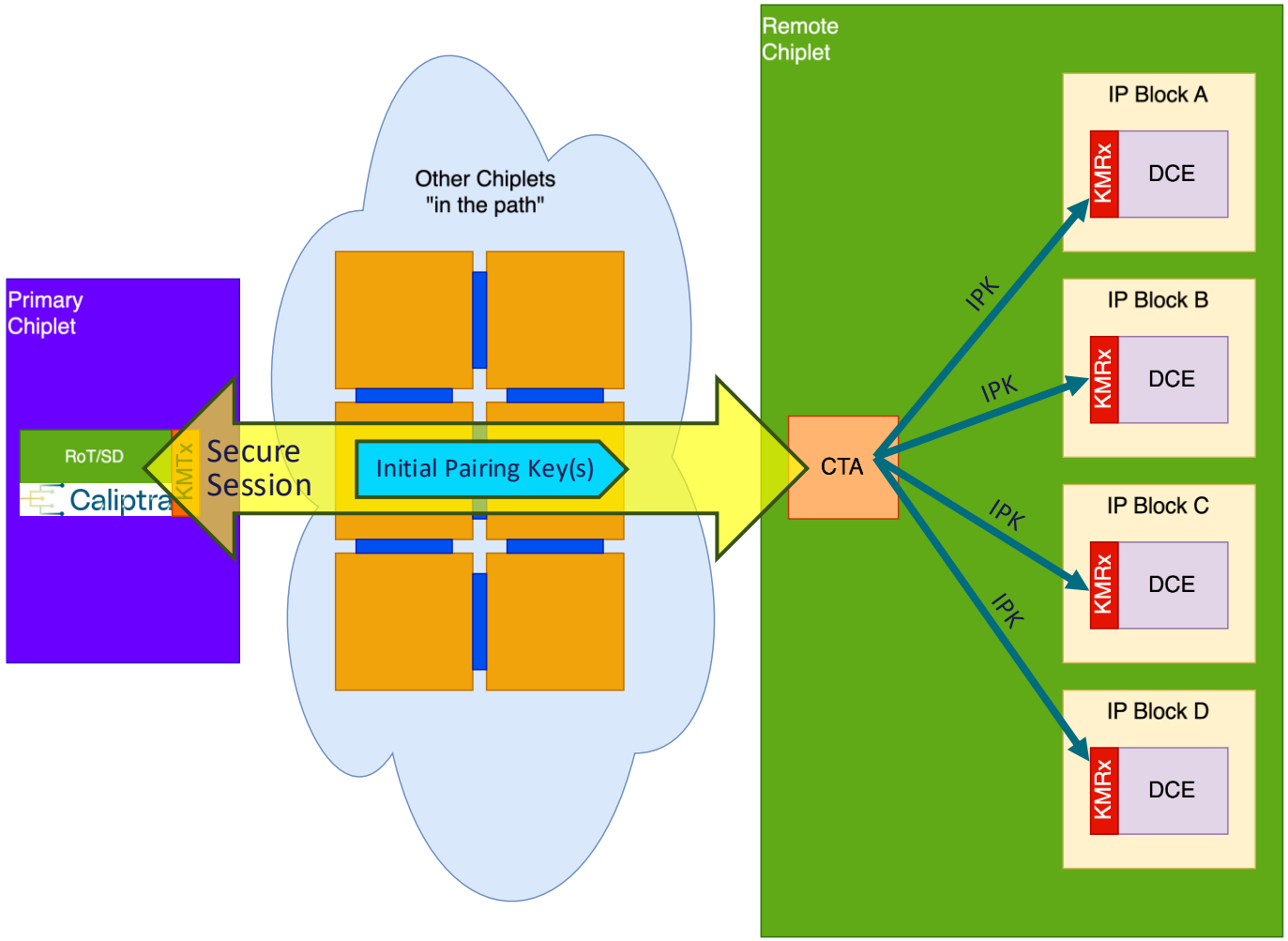
- RoT establishes Initial Pairing Keys to KMRx instances on “remote” chiplets as follows:
  1. RoT establishes secure session to remote CTA

Proposed CTA feature: KMRx Initial Pairing Key (IPK) Establishment



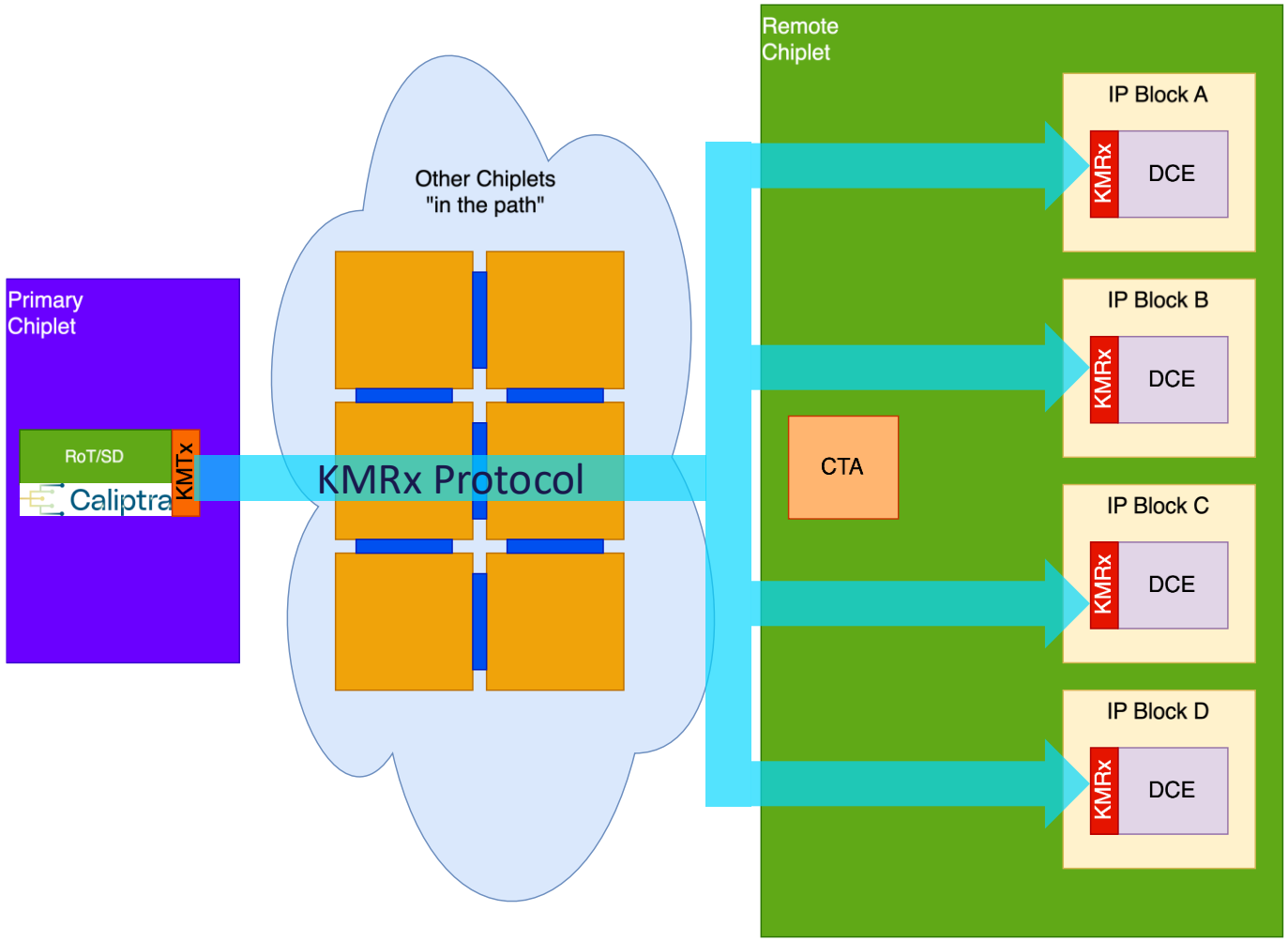
- RoT establishes Initial Pairing Keys to KMRx instances on “remote” chiplets as follows:
  1. RoT establishes secure session to remote CTA
  2. RoT pushes KMRx IPKs to CTA over secure session

Proposed CTA feature: KMRx Initial Pairing Key (IPK) Establishment



- RoT establishes Initial Pairing Keys to KMRx instances on “remote” chiplets as follows:
  1. RoT establishes secure session to remote CTA
  2. RoT pushes KMRx IPKs to CTA over secure session
  3. CTA distributes IPKs to KMRx instances on its local chiplet

Proposed CTA feature: KMRx Initial Pairing Key (IPK) Establishment



- RoT establishes Initial Pairing Keys to KMRx instances on “remote” chiplets as follows:
  1. RoT establishes secure session to remote CTA
  2. RoT pushes KMRx IPKs to CTA over secure session
  3. CTA distributes IPKs to KMRx instances on its local chiplet
  4. RoT/KMTx can now securely push DCE keys to KMRx instances using the fast fabrics/buses/chiplet-to-chiplet interfaces

Call to Action

- Join the “Caliptra as Security Hub” Sub Working Group
  - Biweekly Zoom meeting
- Review the v0.7 spec draft and share feedback
  - Via the OCP Security Hub Google Drive Folder
- We need your expertise:
  - Refine the SoC/SiP key distribution spec
  - Stress-test the threat model
  - Contribute to RTL development
- Help us drive v0.7 to v1.0

Thank You!

Click here to review OCP Trademark Guidelines



Backup: Key Management Security Goals

- Never observable in plaintext
- Only usable in the intended context
- Keys cannot outlive their purpose
- No single compromised layer can expose or control keys
- System behaves consistently during transitions
- Minimal blast radius of on-chip key exposure
- Verifiable / auditable end-to-end solution
- Less error-prone to implement